

ComicCraft – AI Comic Story Creator using Gemini Models

Phase 5 – Project Development Phase

1. Introduction

In Phase 5, the ComicCraft project is developed using the requirements and design prepared in the previous phases. The application takes a user's story idea and uses the Gemini AI model to generate a comic story with:

- Story title
- Characters
- Scenes
- Dialogues
- Comic panel descriptions

2. Technologies Used

- Frontend: HTML, CSS
- Backend: Python, FastAPI
- AI Model: Gemini
- API: Gemini API
- Programming Language: Python

3. Project Development Flow

```
graph TD
    User --> EnterStoryPrompt[Enter Story Prompt]
    EnterStoryPrompt --> Frontend
    Frontend --> FastAPIBackend[FastAPI Backend]
    FastAPIBackend --> GeminiAI[Gemini AI]
    GeminiAI --> GenerateComicStory[Generate Comic Story]
    GenerateComicStory --> DisplayResult[Display Result]
```

User
↓
Enter Story Prompt
↓
Frontend
↓
FastAPI Backend
↓
Gemini AI
↓
Generate Comic Story
↓
Display Result

4. Backend Coding

main.py

```
from fastapi import FastAPI
from pydantic import BaseModel
from google import genai

app = FastAPI()

client = genai.Client(api_key="YOUR_GEMINI_API_KEY")

class StoryRequest(BaseModel):
    prompt: str

@app.post("/generate")
def generate_story(request: StoryRequest):

    instruction = f"""
    Create a simple comic story from this idea:

    {request.prompt}

    Include:
    1. Title
    2. Characters
    3. Scenes
    4. Dialogues
    5. Comic panel descriptions
    """

    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=instruction
    )

    return {
        "story": response.text
    }
```

5. Frontend Coding

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>ComicCraft</title>

    <style>
        body {
            font-family: Arial;
```

```

        text-align: center;
        background: #f5f5f5;
    }

    textarea {
        width: 80%;
        height: 120px;
        padding: 10px;
    }

    button {
        margin: 15px;
        padding: 10px 20px;
        cursor: pointer;
    }

    #result {
        width: 80%;
        margin: auto;
        padding: 20px;
        background: white;
        text-align: left;
    }
</style>
</head>

<body>

    <h1>ComicCraft</h1>
    <h2>AI Comic Story Creator</h2>

    <textarea id="prompt"
        placeholder="Enter your comic story idea..."></textarea>

    <br>

    <button onclick="generateStory()">
        Generate Comic
    </button>

    <div id="result"></div>

    <script>
        async function generateStory() {

            const prompt =
                document.getElementById("prompt").value;

            const response = await fetch("/generate", {
                method: "POST",

                headers: {
                    "Content-Type": "application/json"
                },

```

```

        body: JSON.stringify({
            prompt: prompt
        })
    });

    const data = await response.json();

    document.getElementById("result").innerText =
        data.story;
}
</script>

</body>
</html>

```

6. How the Code Works

Step 1 – User Input

The user enters a simple idea, for example: A boy finds a magical book.

Step 2 – Frontend

The HTML page sends the user's prompt to the FastAPI backend.

Step 3 – Backend

FastAPI receives the story prompt.

Step 4 – Gemini AI

The prompt is sent to the Gemini model.

Step 5 – Story Generation

Gemini generates the title, characters, scenes, dialogues, and panel descriptions.

Step 6 – Output

The generated comic story is displayed on the webpage.

7. Example Output

Input:

A boy finds a magical book.

Output:

Title: The Magical Book

Characters:

1. Arun - a curious boy
2. Maya - his friend

Scene 1:

Arun finds an old book in the library.

Dialogue:

Arun: "What is this strange book?"

Scene 2:

The book starts glowing.

Dialogue:

Maya: "Arun, look! The book is magical!"

Panel Description:

Arun and Maya stand inside the library while the magical book glows brightly.

8. Installation

Install the required packages:

```
pip install fastapi uvicorn google-genai
```

Run the application:

```
uvicorn main:app --reload
```

Then open the application in a browser.

9. Phase 5 Result

The Project Development Phase successfully converts the ComicCraft design into a working application.

- Accept a story prompt
- Connect to Gemini AI
- Generate a comic story
- Create characters and scenes
- Generate dialogues
- Generate comic panel descriptions
- Display the result to the user

Phase 5 is therefore the implementation stage where the actual ComicCraft application is developed.